# Homework 7 - Making Data Types

**Course:** CIS 284 - Computer Programming 2 - Fall 2026
**Instructor:** Dr. Aaron Mininger

## Submission Instructions

**Due Date:** Nov 12 at 11:59pm

Zip up all java files and upload to Canvas. **Make sure each file has your name at the top**

For 2 bonus points, answer the following questions (as a **comment** on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear in the directions? Do you have any other feedback?

Your responses will be helpful feedback for refining these assignments in future courses.

---

## Assignment Overview

This assignment will help you practice writing data types through two applications. The first is making a starfield animation via a particle system. The second is re-writing your Stats program to make a proper `DataSet` class.

## Grading Information

*If you can't complete everything, it is better to submit partial work than miss the deadline.*

| Points | Name | Description |
|---|---|---|
| 25 | Effort | Do the programs compile and run? Was a reasonable attempt made? |
| 5 | Style | Is the code formatted well? Is it readable? Are variables named well? |
| 25 | Part 1 | Correct implementation of `Star` class. |
| 40 | Part 2 | Correct implementation of `DataSet` API. |
| 5 | Part 2 | All methods in `DataSet` are commented. |

Note: Your output does not need to match my examples word for word. Feel free to add additional messages or personalize it, as long as it satisfies the requirements.

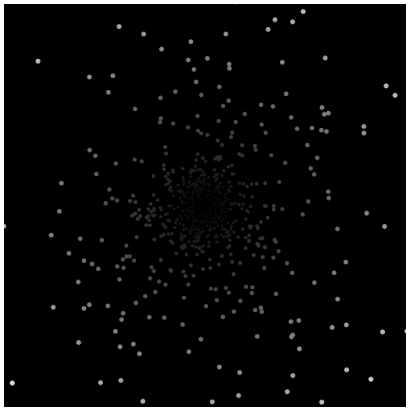
## 1. Starfield

In this assignment, you are going to create a **particle system**: a simulation involving many individual particles, or small objects, that each contain their own state information.

The internal state typically includes things like the particle's current position and information needed to move and draw the particle.

Particle systems are used in computer graphics to draw things such as snow, fire, water, smoke, etc.

The particle system you will implement will be a starfield animation.



To do this, we will define 2 classes:

- **`Star.java`** (an individual star particle)
- **`Starfield.java`** (the program that creates/runs all particles)

File **`Starfield.java`**

First, open up **`Starfield.java`** and study the code inside. The main method is provided for you. (You do not have to edit this file)

It does 3 things:

1. Initializes the canvas (x,y scale from -1 to 1)
2. Creates N **`Star`** objects in an array
3. Runs the animation loop (update stars, draw stars, pause)

Note that we are using double buffering: at the start of every loop we clear the canvas, and at the end we call **`show()`**.

B. **`Star.java`**

Now, open up **`Star.java`**. You can see that there are 3 methods you must implement:

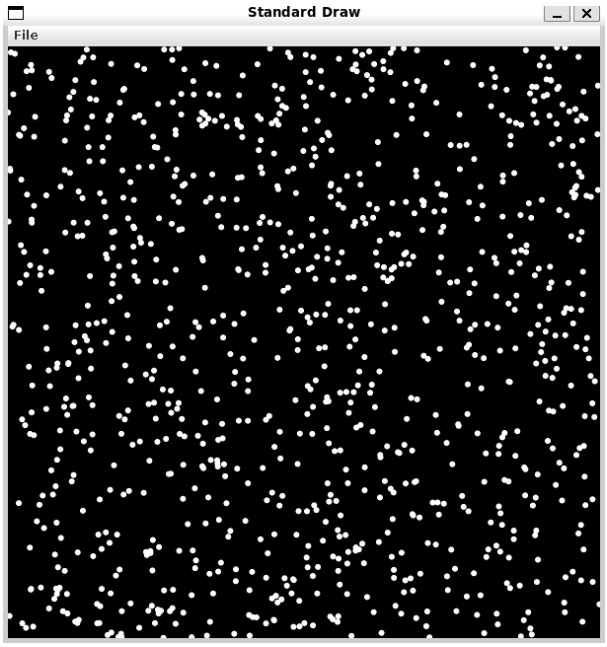
- **`Star()`** - the default constructor
- **`void update()`** - update the Star's position/age
- **`void draw()`** - draw the star to the canvas

**Step 1:** Define position and draw the particle

1. Create instance variables **`x`** and **`y`** that will be the star's position.
2. Create instance variable **`Color color`**
3. In the constructor, initialize **`x`** and **`y`** to **`2*Math.random() - 1`** (random values from -1 to 1)
4. In the constructor, set **`color`** to **`Color.white`**
5. In the draw method, draw a filled circle with radius 0.01 using **`color`** and at position **`x`** and **`y`**

(The update method will stay empty)

If you compile **`Star.java`** and run **`Starfield.java`**, it should look something like this:




Make sure this is working before you continue.

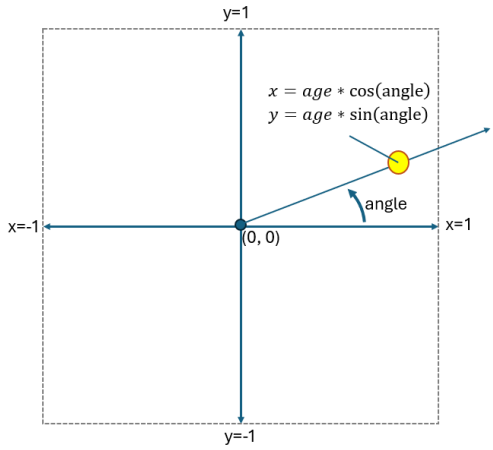
**Step 2:** Defining the particle's age

Think of each Star travelling on a ray that starts at the origin and moves outward in a straight line until it goes off the window.

1. Add an instance variable **`angle`** which will define the direction the star moves in.
2. In the constructor, set **`angle`** to be a random number from 0 to 2*Pi
3. Define an instance variable **`age`** and initialize it to 0 in the constructor

The **`age`** variable will gradually increase from 0 (start) to 1 (end).

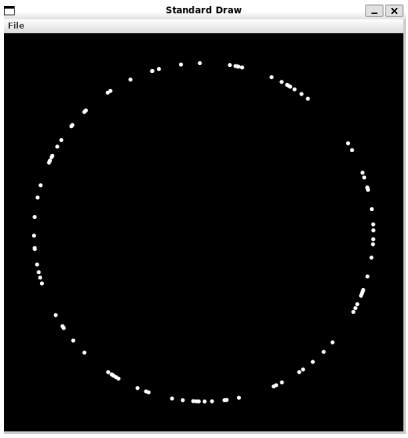
- At **`age == 0`**, the particle will be at the origin (0, 0)
- At **`age == 1`**, the particle will be at the end of the ray (off the screen).




Modify the update method as follows:

1. Increase **`age`** by 0.01
2. If **`age > 1`**, reset it back to 0
3. Set x = *age * cos(angle)*
4. Set y = *age * sin(angle)*

It should look something like this:



**Step 3:** Visual Tweaks

To make the final result look good, there are a few tweaks to make.

First, all the particles are starting at age 0, which creates a ring. We want to randomize this.

- In the constructor: initialize **`age = Math.random();`**

Second, you don't want the position to move linearly with age. You want the stars to start slow, then speed up. So we add a **`scale`** value that starts slow and speeds up as age increases:




- Inside **`update()`**, set a local variable **`double scale = age / (12 - 11*age);`**

We are going to base everything on this scale factor. Modify **`update()`** so it does the following:

1. Set *x = scale * cos(angle) * 1.5*
2. Set *y = scale * sin(angle) * 1.5*
3. Set the color to a gray value with brightness = 255 * scale
4. Add an instance variable **`radius`**, and set radius = 0.02*scale (Use this radius when drawing the circle).

This should complete the starfield animation.

## Summary

1. In the constructor, you should start with a random age and angle.
2. In **`update()`**, you should update age, then compute the scale, then use the scale to compute x, y, color, and radius values
3. In **`draw()`**, you should draw a circle using color, x, y, and radius

## Explore Further

You can earn some extra credit by modifying this starfield animation further. For Example:

- Add additional random parameters to vary the speeds, colors, etc.
- Make it interactive (modify speed using keyboard keys)
- Or create a different particle system entirely.

## 2. Stats Program

For one final time, we are going to revist our Stats project from previous homeworks. This time, we are going to rewrite it using two classes: **`DataSet.java`** and **`StatsProgram.java`**

### `StatsProgram.java`

Open this file and examine the code. I have fully implemented this program for you. Note that I have made this class a proper data type: none of the methods are static and it contains a **`dataset`** as a member variable. All the methods can access this variable, so all of the methods end up being void and take no parameters.

### `DataSet.java`

The **`StatsProgram`** class depends on a **`DataSet`** class, which you will implement.

Create the file **`DataSet.java`** and implement the following API:

| `public class DataSet` | |
|---|---|
| `DataSet()` | *Creates a new data set with 0 values* |
| `void load(File file)` | *Loads the values from the file* |
| `void save(File file)` | *Writes the values to the given file* |
| `int size()` | *Returns the number of values in the data set* |
| `double get(int i)` | *Returns the value at index i* |
| `void add(double num)` | *Adds the given value to data set* |
| `void remove(int i)` | *Removes the value at index i* |
| `double min()` | *Returns the minimum value in the data set* |
| `double max()` | *Returns the maximum value in the data set* |
| `double sum()` | *Returns the sum of all values in the data set* |
| `double mean()` | *Returns the mean of the data set* |
| `double var()` | *Returns the variance of the data* |

**Notes:**

1. The **`DataSet`** class should contain the array of doubles as a *private* instance variable
2. The **`load`** and **`save`** methods should catch the file not found exceptions internally.
3. **Every method should have a brief comment above describing what it does**
4. You are encouraged to adapt your previous code instead of starting from scratch.

You will be graded on how well your **`DataSet`** class implements this API.